SESSION 1 – C PROGRAMS

Name: Arun.p
Register No: 192372013

1. First Palindromic String

C Program

```
#include <stdio.h>
#include <string.h>

int main() {     char words[][20] = {"abc", "car", "ada",
"racecar", "cool"};     int n = 5;

    for (int i = 0; i < n; i++) {         int left = 0, right
= strlen(words[i]) - 1, flag = 1;         while (left < right)
{             if (words[i][left] != words[i][right]) {
flag = 0;                 break;             }
left++;             right--;         }         if (flag) {
printf("First palindrome: %s\\n", words[i]);
return 0;         }     }     printf("No palindrome
found\\n");     return 0; }
```

Output Screenshot

```
C Program Output
```

```
C:\Lab> gcc program.c -o program
C:\Lab> program

First palindrome: ada
```

Output: First palindrome: ada

2. Count Common Elements

C Program

```
#include <stdio.h>

int contains(int a[], int n, int x) {
for (int i = 0; i < n; i++)         if
(a[i] == x) return 1;     return 0; }

int main() {     int nums1[] = {2,
3, 2};     int nums2[] = {1, 2};
int n = 3, m = 2;     int answer1
= 0, answer2 = 0;
```

Name: Arun.p Register No: 192372013

```
    for (int i = 0; i < n; i++)         if
(contains(nums2, m, nums1[i])) answer1++;

    for (int i = 0; i < m; i++)         if
(contains(nums1, n, nums2[i])) answer2++;

    printf("[%d, %d]\\n", answer1, answer2);
return 0; }
```

Output Screenshot

```
C Program Output
```

```
C:\Lab> gcc program.c -o program
C:\Lab> program

[2, 1]
```

Output: [2, 1]

3. Sum of Squares of Distinct Counts

C Program

```
#include <stdio.h>

int main() {     int nums[]
= {1, 2, 1};     int n = 3,
sum = 0;

    for (int i = 0; i < n; i++) {         int
distinct[100], d = 0;         for (int j = i; j < n;
j++) {             int found = 0;             for (int
k = 0; k < d; k++)                 if (distinct[k] ==
nums[j]) found = 1;

            if (!found) distinct[d++] = nums[j];
sum += d * d;         }     }

    printf("Sum = %d\\n", sum);
return 0; }
```

Output Screenshot

```
C Program Output
```

```
C:\Lab> gcc program.c -o program
C:\Lab> program

Sum = 15
```

Output: Sum = 15

4. Equal Pairs with Product Divisible by k

Name: Arun.p Register No: 192372013

C Program

```
#include <stdio.h>

int main() {     int nums[] = {3, 1, 2,
2, 2, 1, 3};     int n = 7, k = 2, count
= 0;

    for (int i = 0; i < n; i++) {          for (int j = i
+ 1; j < n; j++) {              if (nums[i] == nums[j] &&
(i * j) % k == 0)                  count++;          }
}

    printf("Number of valid pairs = %d\\n", count);
return 0; }
```

Output Screenshot

C Program Output

```
C:\Lab> gcc program.c -o program
C:\Lab> program

Number of valid pairs = 4
```

Output: Number of valid pairs = 4

5. Find Maximum with Least Time Complexity

C Program

```
#include <stdio.h>

int main() {     int a[] = {1,
2, 3, 4, 5};     int n = 5;
int max = a[0];

    for (int i = 1; i < n; i++)
if (a[i] > max) max = a[i];

    printf("Maximum = %d\\n", max);
return 0; }
```

Output Screenshot

C Program Output

```
C:\Lab> gcc program.c -o program
C:\Lab> program

Maximum = 5
```

Output: Maximum = 5

Name: Arun.p Register No: 192372013

6. Maximum Element After Sorting

C Program

```
#include <stdio.h>

void sort(int a[], int n) {     for (int i =
0; i < n - 1; i++)          for (int j = 0; j
< n - 1 - i; j++)              if (a[j] > a[j
+ 1]) {                  int t = a[j];
a[j] = a[j + 1];                  a[j + 1] =
t;              } }

int main() {     int a[] = {3,
3, 3, 3, 3};     int n = 5;

    if (n == 0) {
printf("List is empty\\n");
return 0;     }

    sort(a, n);     printf("Maximum =
%d\\n", a[n - 1]);     return 0; }
```

Output Screenshot

```
C Program Output
```

```
C:\Lab> gcc program.c -o program
C:\Lab> program

Maximum = 3
```

Output: Maximum = 3

7. Remove Duplicate Elements

C Program

```
#include <stdio.h>

int main() {     int a[] = {3, 7, 3, 5,
2, 5, 9, 2};     int n = 8, first = 1;

    printf("Unique elements = [");     for
(int i = 0; i < n; i++) {         int
duplicate = 0;          for (int j = 0; j < i;
j++)              if (a[i] == a[j]) duplicate
= 1;

        if (!duplicate) {
if (!first) printf(", ");
printf("%d", a[i]);              first
= 0;         }     }
printf("]\\n");     return 0; }
```

Output Screenshot

Name: Arun.p Register No: 192372013

```
C Program Output
```

```
C:\Lab> gcc program.c -o program
C:\Lab> program

Unique elements = [3, 7, 5, 2, 9]
```

Output: Unique elements = [3, 7, 5, 2, 9]

8. Bubble Sort and Binary Search

C Program

```
#include <stdio.h>

void bubbleSort(int a[], int n) {     for (int i = 0; i < n -
1; i++)         for (int j = 0; j < n - 1 - i; j++)
if (a[j] > a[j + 1]) {                 int t = a[j]; a[j] =
a[j + 1]; a[j + 1] = t;             } }

int binarySearch(int a[], int n, int key) {
int low = 0, high = n - 1;     while (low <=
high) {         int mid = (low + high) / 2;
if (a[mid] == key) return mid;         if
(a[mid] < key) low = mid + 1;         else
high = mid - 1;     }     return -1; }

int main() {     int a[] = {3, 4, 6, -9,
10, 8, 9, 30};     int n = 8, key = 10;

    bubbleSort(a, n);     int pos =
binarySearch(a, n, key);

    printf("Sorted array: ");     for (int i = 0; i <
n; i++) printf("%d ", a[i]);     printf("\\n");

    if (pos != -1)         printf("Element %d is found at sorted position
%d\\n", key, pos + 1);     else         printf("Element %d is not found\\n",
key);

    printf("Bubble Sort Time Complexity: O(n^2)\\n");
printf("Binary Search Time Complexity: O(log n)\\n");
return 0; }
```

Output Screenshot

```
C Program Output
```

```
C:\Lab> gcc program.c -o program
C:\Lab> program

Sorted array: -9 3 4 6 8 9 10 30
Element 10 is found at sorted position 7
Bubble Sort Time Complexity: O(n^2)
Binary Search Time Complexity: O(log n)
```

Output: Sorted array: -9 3 4 6 8 9 10 30
Element 10 is found at sorted position 7
Bubble Sort Time Complexity: O(n^2)

Name: Arun.p Register No: 192372013

Binary Search Time Complexity: O(log n)